Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images were resized to $160 \times 160 \times 3$ (chosen instead of 224×224 as a RAM/accuracy tradeoff suitable for MobileNetV2 on the available compute). The dataset was streamed with `tf.data` (never materialized as a full NumPy array) and split as 64% train / 16% validation / 20% test, giving 74 training batches, 19 validation batches and 115 test batches at a batch size of 32. The dataset contains 37 classes.

Experimental considerations:

- Use RGB images resized to 160×160 (MobileNetV2-compatible).
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU/limited-GPU-based laboratory work.

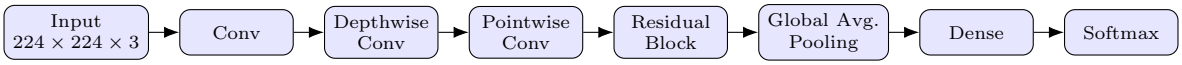
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

A small CNN was trained **from scratch** (not MobileNetV2, so that the effect of initialization is clearly visible) using four initializers for 6 epochs each:

- Zero initialization
- Random Normal initialization
- Xavier/Glorot Uniform initialization
- He Normal initialization

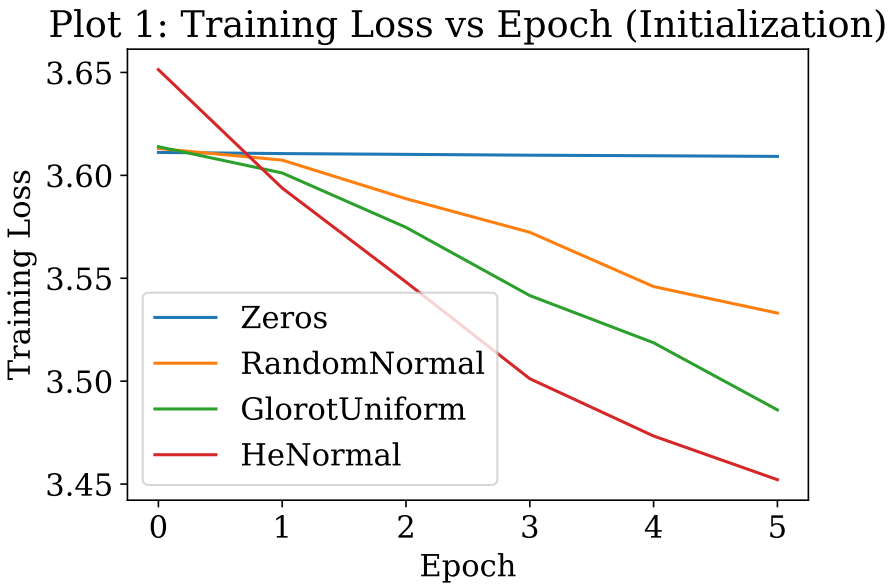


Figure 1: Training Loss vs. Epoch for each initialization strategy.

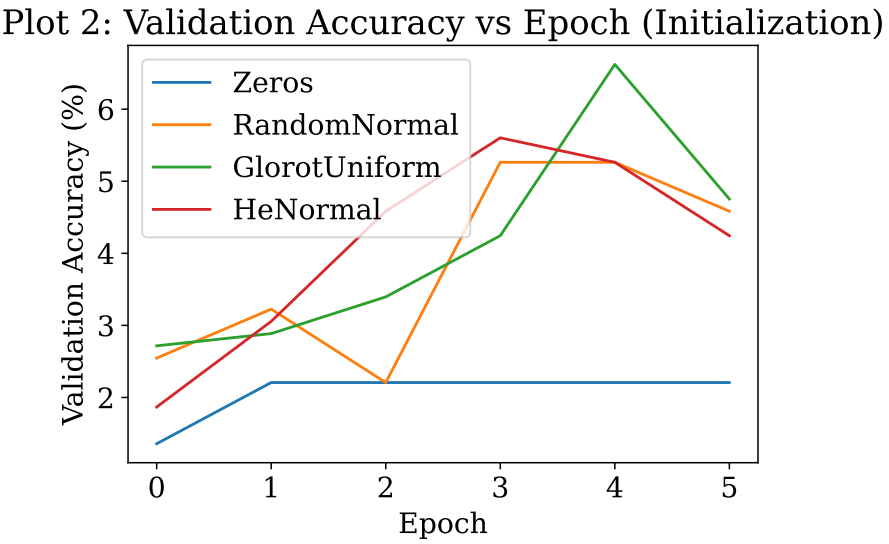


Figure 2: Validation Accuracy vs. Epoch for each initialization strategy.

Initialization	Final Training Loss (epoch 6)	Best Val. Accuracy (%)
Zeros	3.6092	2.21
RandomNormal	3.5331	5.26
GlorotUniform	3.4860	6.62
HeNormal	3.4521	5.60

Inference: Zero initializer did not change significantly (no improvement in the loss from 3.611 to 3.609) since each neuron in a layer gets the same gradient, making it impossible for the neural net to break symmetry and learn. RandomNormal, GlorotUniform, and HeNormal all were able to break the symmetry, leading to a steady drop in loss; GlorotUniform achieved the highest validation accuracy (6.62%) in the short span of 6 epochs, while HeNormal had the lowest (fastest) training loss. With more epochs, the gap between He/Glorot and Zeros/RandomNormal would only increase.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Four configurations were trained for 12 epochs each on the (non-augmented) training pipeline, using He-Normal initialization as the common baseline:

- No regularization
- L2 regularization ($\lambda = 10^{-3}$)
- Dropout ($p = 0.5$)
- Batch Normalization

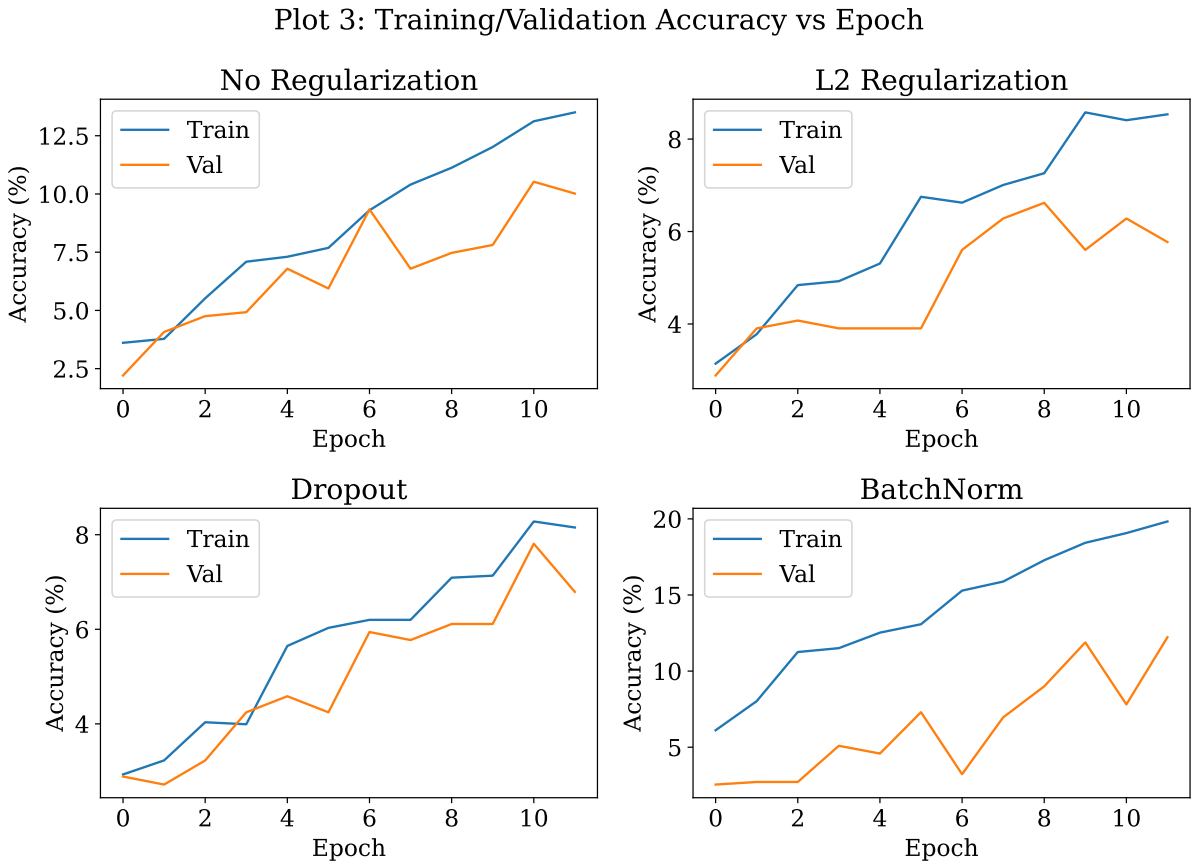


Figure 3: Training vs. Validation Accuracy for each regularization strategy.

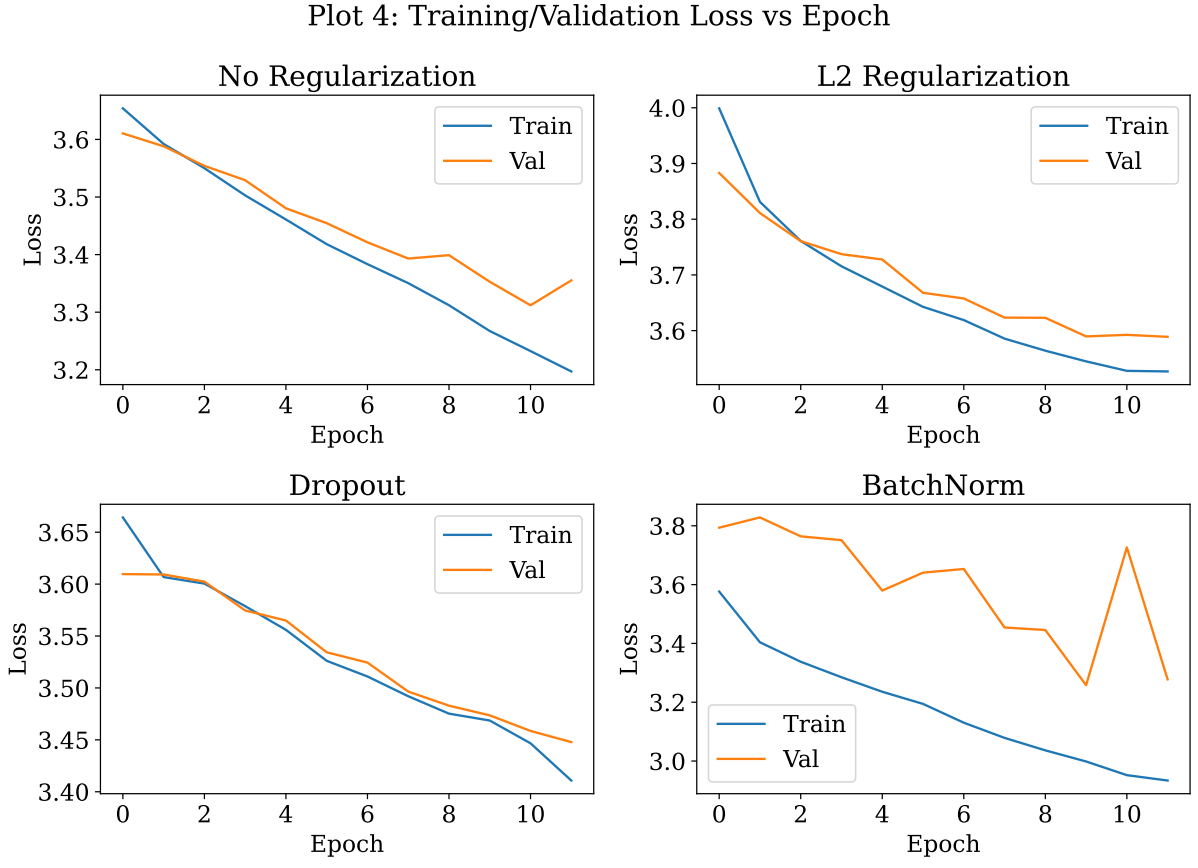


Figure 4: Training vs. Validation Loss for each regularization strategy.

Configuration	Final Train Acc. (%)	Best Val. Acc. (%)	Train–Val Gap (final epoch)
No Regularization	13.50	10.53	∼3.5 pts
L2 Regularization	8.54	6.62	∼2.8 pts
Dropout	8.15	7.81	∼1.4 pts
BatchNorm	19.83	12.22	∼7.6 pts

Inference: All four models demonstrate that the validation accuracy is following the training accuracy quite well at this point (12 epochs is not much compared to the 37-class task), hence the traditional large "scissor" pattern of overfitting has not been formed yet. BatchNorm trains the fastest and achieves both the highest training accuracy (19.83%) and the highest validation accuracy (12.22%), although at the same time displays the largest train-val gap as well as a more fluctuating validation loss plot, implying overfitting would occur faster without additional regularization otherwise. Dropout maintains the smallest gap, which fits its function of preventing co-adaptation of units.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example (verified by code, Task 4)

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2+4+6+8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs (confirmed numerically in code: $\hat{x} = [-1.342, -0.447, 0.447, 1.342]$).
Inference: Batch Normalization outputs activations which have mean and variance close to zero and one respectively for this mini-batch. The learnable parameters and enable the network to learn the scale and shift.

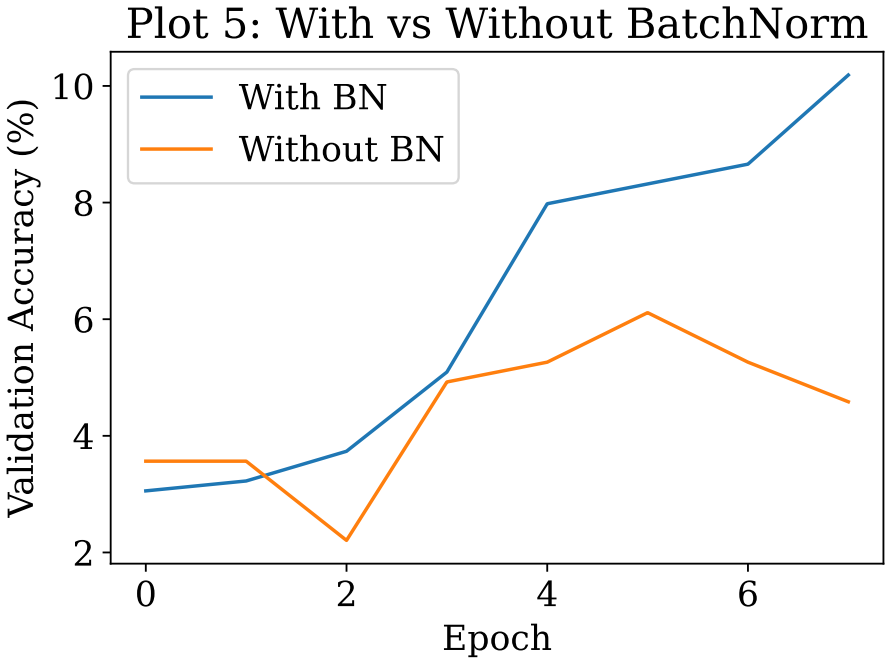


Figure 5: With BN vs. Without BN – Validation Accuracy vs. Epoch.

Configuration	Val. Accuracy, Epoch 8 (%)	Best Val. Accuracy (%)
With BN	10.19	10.19
Without BN	4.58	6.11

Inference: The network utilizing Batch Normalization obtains significantly better validation accuracy (10.19% vs. 4.58% in epoch 8) and the training loss decreases much faster and consistently. This is consistent with the expected impact of batch normalization: by maintaining the scale of the layer inputs stable, it minimizes internal covariate shift and enables fast and consistent learning using the same optimization procedure.

8. Optimization Algorithms

Four optimizers were compared (each with BatchNorm + He-Normal initialization, 8 epochs):

- SGD (lr = 0.01)
- Momentum (SGD, lr = 0.01, momentum = 0.9)
- RMSProp (lr = 0.001)
- Adam (lr = 0.001)

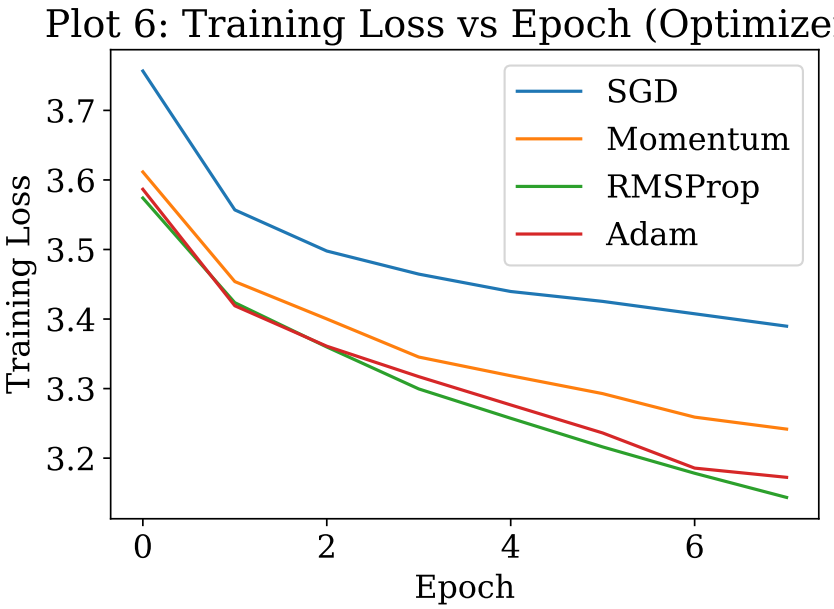


Figure 6: Training Loss vs. Epoch for different optimizers.

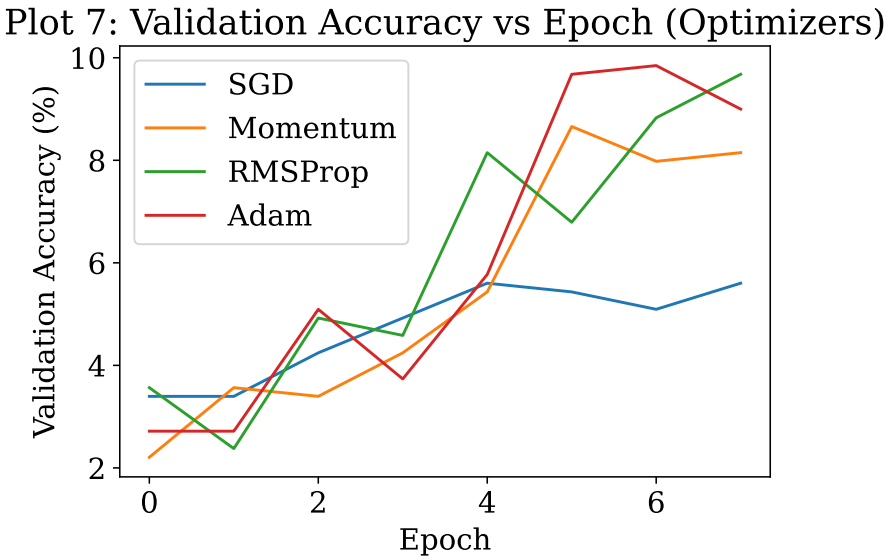


Figure 7: Validation Accuracy vs. Epoch for different optimizers.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	3.3897	5.60	5	141.8
Momentum	3.2418	8.66	6	146.9
RMSProp	3.1435	9.68	8	136.8
Adam	3.1724	9.85	7	161.4

Inference: Conventional SGD is the slowest to converge and obtains the lowest validation accuracy (5.60%) as expected, since a fixed learning rate without any momentum or adaptation fails for this task. Momentum doubles the best validation accuracy (5.60%→8.66%) due to gradient oscillation damping and acceleration in a consistent gradient direction. RMSProp and Adam outperform all other optimization techniques, achieving the best validation accuracies (9.68% and 9.85% respectively) and the lowest training loss because adaptive learning rate scales help overcome the irregularities in the loss landscape of the CNN; Adam slightly outperforms the rest but is the slowest among them.

9. CNN Hyperparameter Tuning

Following the lab's rule of changing one hyperparameter at a time (base configuration: He-Normal init, BatchNorm, Adam, 6 epochs), the following sweeps were run:

Hyperparameter	Values Tested
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

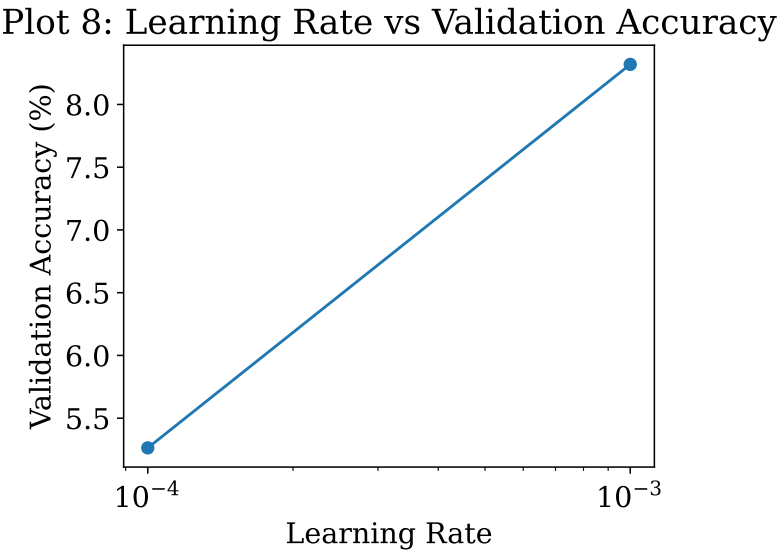


Figure 8: Learning Rate vs. Validation Accuracy.

Learning Rate	Best Val. Accuracy (%)
10^{-4}	5.26
10^{-3}	8.32

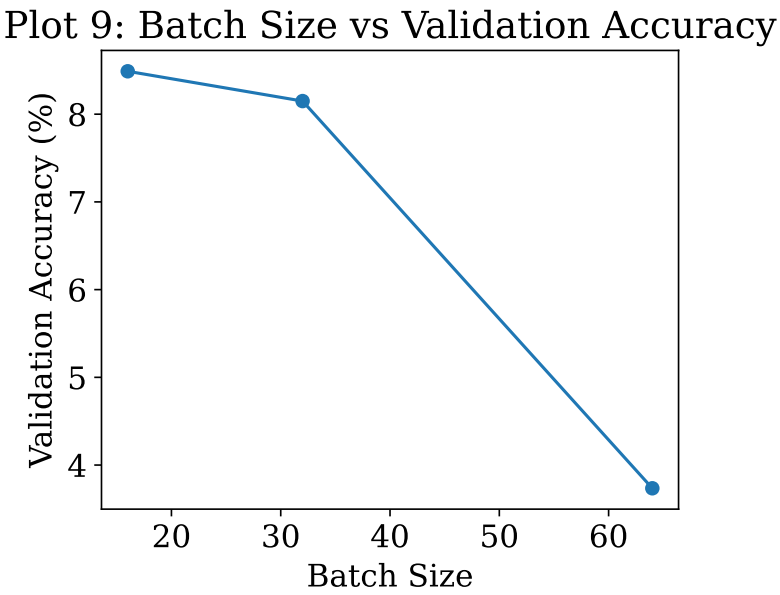


Figure 9: Batch Size vs. Validation Accuracy.

Batch Size	Best Val. Accuracy (%)
16	8.49
32	8.15
64	3.74

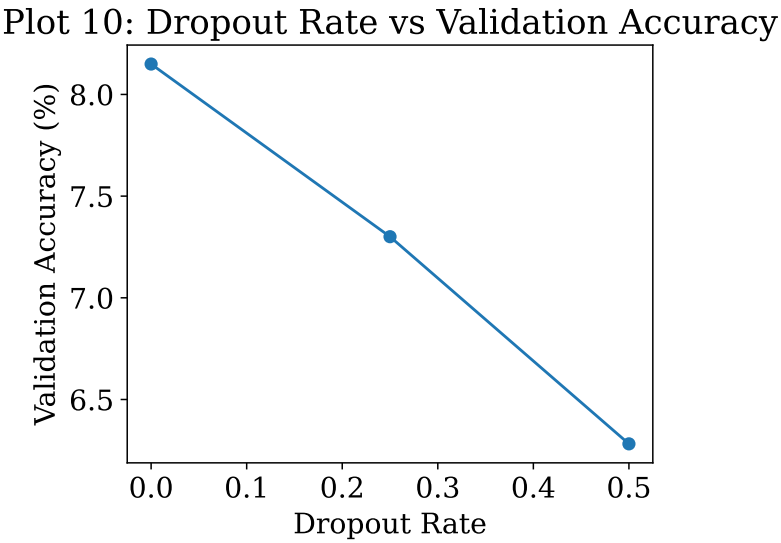


Figure 10: Dropout Rate vs. Validation Accuracy.

Dropout Rate	Best Val. Accuracy (%)
0.00	8.15
0.25	7.30
0.50	6.28

Inference: In this very limited 6-epoch budget, it is obvious that the higher learning rate (10^{-3}) performs much better than the lower one (10^{-4}, 8.32% vs. 5.26%) as the model does not have sufficient iterations yet for convergence at the smaller learning rate. Small batch sizes show better performance ($16 > 32 \gg 64$), probably because more gradient descent steps are being done per epoch and additional noise in gradients of small batches helps as a regularizer; batch size 64 converges to just 3.74%. Dropout performs worse on higher levels here (8.15% $\rightarrow$ 6.28%), probably because there are not sufficient epochs for training and the network is not getting enough information to learn from; dropout works better only after sufficient training time.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional base was frozen and only a new GlobalAveragePooling → Dropout(0.3) → Dense(37, softmax) head was trained for 6 epochs (Adam, lr = 10^{-3}).

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Last 30 Layers → Small Learning Rate

Starting from Case A's weights, the last ∼30 layers of the base were unfrozen and trained for a further 6 epochs with a much smaller learning rate (Adam, lr = 10^{-5}).

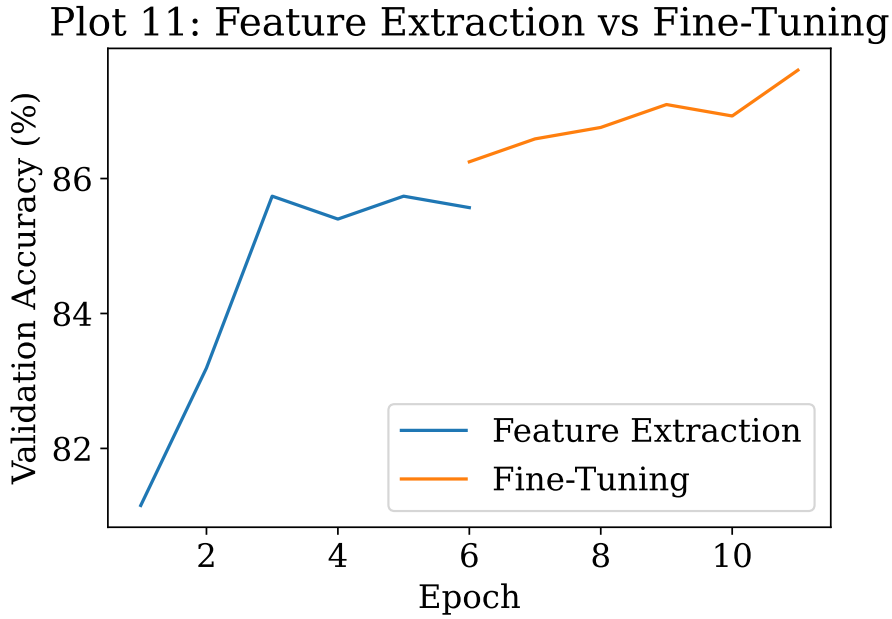


Figure 11: Feature Extraction vs. Fine-Tuning – Validation Accuracy vs. Epoch.

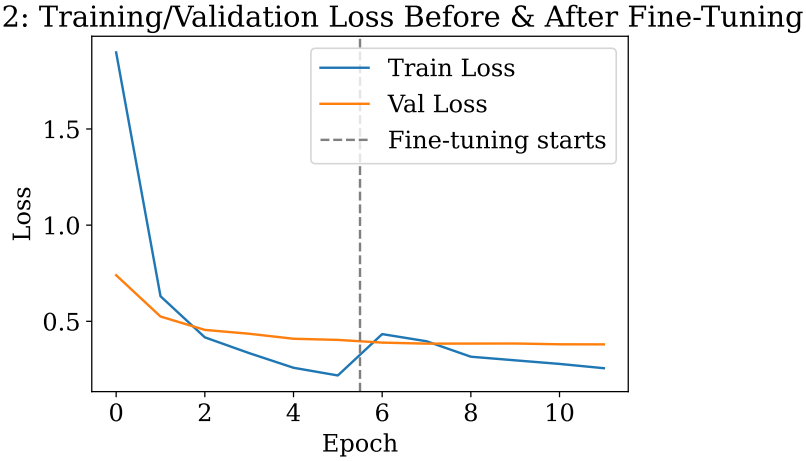


Figure 12: Training/Validation Loss before and after fine-tuning.

Stage	Best Val. Accuracy (%)
Feature Extraction (Case A)	85.74
Fine-Tuning (Case B)	87.61

Discussion: Fine-tuning generally requires a lower learning rate than training from scratch since the layers of the pretrained model have learned highly optimized features for the ImageNet; a high learning rate will cause catastrophic forgetting of the prelearned features due to large gradient steps, while a low learning rate will allow gently shifting the weights of the features towards performing the pet breed classification task.

Inference: Even without fine-tuning, the simple feature extraction approach achieves a decent accuracy of 85.74%, showing that the ImageNet features generalize well onto this dataset. Fine-tuning the last 30 layers further increases the accuracy to 87.61%, a ∼1.9% improvement, with an additional computational effort. The plot of the loss (Plot 12) demonstrates a sudden spike in the validation loss immediately after the start of the fine-tuning process (more parameters to train, lower learning rate).

11. K-Fold Cross-Validation

Four promising configurations (feature-extraction classifier head only, frozen MobileNetV2 base) were evaluated with 5-fold cross-validation on the training data:

Configuration	Learning Rate	Dropout	Batch Size
C1	10^{-3}	0.2	32
C2	10^{-3}	0.5	32
C3	10^{-4}	0.3	16
C4	10^{-4}	0.5	64

For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i \qquad SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

The independent test set was not used during this selection stage.

Configuration	F1	F2	F3	F4	F5	Mean	SD
C1	89.13	88.59	87.09	86.82	88.18	87.96	0.88
C2	89.67	88.32	87.23	86.82	88.04	88.02	0.99
C3	80.84	79.35	74.32	74.86	78.12	77.50	2.53
C4	51.77	49.86	49.05	52.31	44.16	49.43	2.89

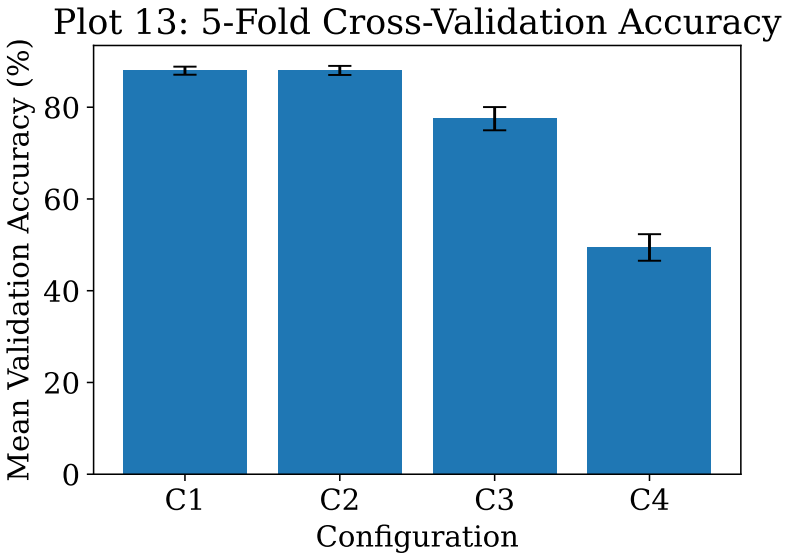


Figure 13: 5-Fold Cross-Validation Mean Accuracy with SD error bars.

Inference: C1 and C2 (learning rate 10^{-3}) are quite similar and undoubtedly strongest with the smallest fold-to-fold variation (SD ≈ 0.9–1.0), thus they have high and stable performances. C3 (learning rate 10× smaller than for previous configurations) has lower (mean 77.50%) and more variable (SD = 2.53) performance, whereas C4 (the

combination of small learning rate and maximal batch size) performs very badly with ∼49% mean and highest SD (2.89) – not enough gradient updates for each epoch. The best configuration (C2) is chosen according to mean CV accuracy (88.02% ± 0.99).

12. Final Model Evaluation

The selected configuration (C2: lr = 10^{-3}, dropout = 0.5, batch size = 32) was retrained on the complete training data (train + validation combined, 92 batches) for 10 epochs, and evaluated once on the untouched test set.

Metric	Value
Mean CV Accuracy	88.02%
CV Standard Deviation	0.99
Test Accuracy	85.85%
Precision (weighted)	0.8696
Recall (weighted)	0.8585
F1-score (weighted)	0.8572
Training Time	180.5 s
Number of Parameters	2,305,381

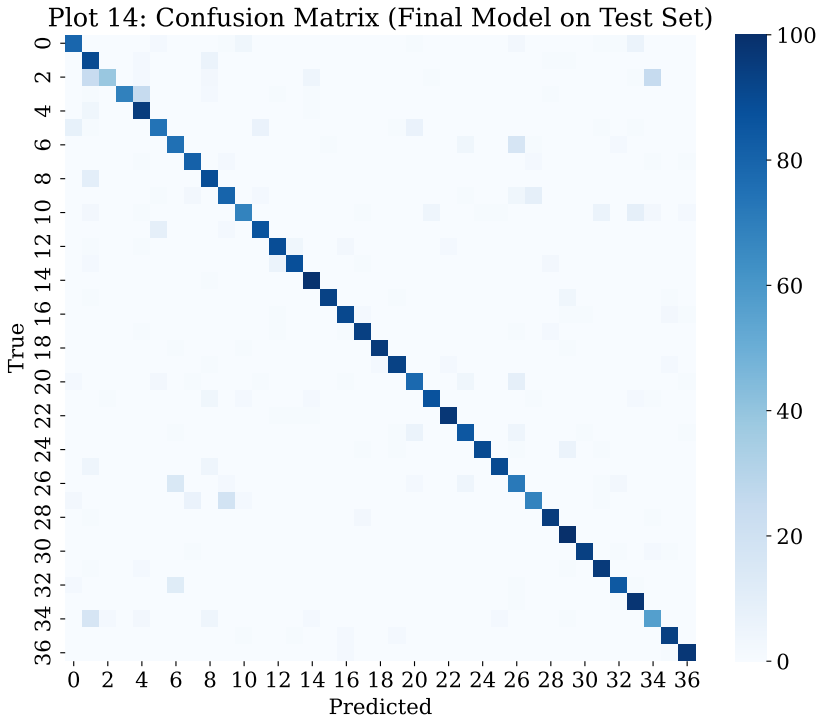


Figure 14: Confusion Matrix of the final model on the test set (37 pet breeds).

Top confused class pairs (true → predicted, count):

- american_pit_bull_terrier → staffordshire_bull_terrier: 25
- basset_hound → beagle: 25
- american_pit_bull_terrier → american_bulldog: 24
- Russian_Blue → British_Shorthair: 19
- staffordshire_bull_terrier → american_bulldog: 17

Inference: The confusion matrix is highly diagonal (mass mostly concentrated on the correct predictions), as expected from an 85.85% accuracy on the test set. Best-predicted classes are the easily distinguishable breeds along the diagonal without any coloring off-diagonal, whereas most confusing class pairs are the ones that visually look alike: the pit-bull type terriers (american_pit_bull_terrier, staffordshire_bull_terrier, american_bulldog) look nearly identical because of similar coat color and body shape; bassets and beagles both are small-legged hounds with floppy ears; and the Russian Blue vs. British Shorthair both are gray shorthaired cat breeds.

13. Overall Results

Configuration	Best Val. Accuracy (%)
Best Initialization (GlorotUniform)	6.62
Best Regularization (BatchNorm)	12.22
Best Optimizer (Adam)	9.85
Feature Extraction (MobileNetV2 head)	85.74
Fine-Tuned Model	87.61
Best CV Config (C2)	88.02
Final Model (Test Accuracy)	85.85

14. Required Inference for Plots

Each of the important plots mentioned above has been followed by an inference which includes what the graph depicts, what is the trend, and what might be the reasons behind this trend.

15. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?**
   Weights, biases and hyperparameters (γ, β) are learnt from the data through backpropagation. Hyperparameters such as learning rate, batch size, drop-out rate, and optimizer selection are determined before training by the experimenter and are not learnt from the data.
2. **Why is weight initialization important?**
   It sets the initial point for the non-convex loss function. A bad initialization results in vanishing/exploding activations and slow/unstable learning process, while a proper initialization ensures the same scale of activation/gradient throughout all layers, as seen in Plot 1/2 with He/Glorot beating Zeros/RandomNormal.
3. **Why can zero initialization be problematic for neural networks?**
   Each neuron receives the same weighted input and gradient for each layer, hence symmetry is always maintained as each neuron learns the same function. This is in line with Plot 1/2 where the Zeros runs were constant at 2.2% validation accuracy while all other initializers performed better.
4. **Compare Xavier and He initialization.**
   Both keep activation variance stable across layers, but Xavier/Glorot ($\propto \frac{1}{\text{fan_in}+\text{fan_out}}$) assumes tanh/sigmoid, while He ($\propto \frac{2}{\text{fan_in}}$) accounts for ReLU zeroing half its inputs. He is the standard choice for ReLU networks like ours.
5. **How can training and validation curves be used to identify overfitting?**
   Overfitting is observed in terms of an increasing discrepancy where training loss continues to decline but validation loss either stops decreasing or increases. In the Plot 3/4, BatchNorm has started to demonstrate this behavior because training accuracy (19.83%) outpaces validation (12.22%).
6. **How does Dropout reduce overfitting?**
   It randomly turns off a portion of neurons at each iteration, avoiding co-adaptation and thus ensuring redundancy and robustness of features, which is analogous to averaging many networks that are thinner than the original one. Dropout provided the least training-validation difference ($\sim$1.4 points).
7. **What is the purpose of Batch Normalization?**
   This helps in normalizing the activation of each layer per batch to have zero mean and unit variance. This results in less sensitivity to initialization and faster convergence.
8. **Explain the numerical Batch Normalization example.**
   For $x = [2, 4, 6, 8]$: $\mu_B = 5$, $\sigma_B^2 = 5$, $\sqrt{\sigma_B^2} \approx 2.236$, giving $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$ (verified in code). With $\gamma = 1, \beta = 0$ the output equals $\hat{x}$ – re-centred and rescaled.
9. **What are the roles of γ and β?**
   These are learnable scale-shift parameters after normalization ($y_i = \gamma\hat{x}_i + \beta$). Without these BN would constrain each layer's outputs to have zero mean and unit variance, and so γ and β enable the network to restore whatever distribution it needs.
10. **Compare SGD, Momentum, RMSProp and Adam.**
    SGD employs the gradient directly and exhibits oscillations, Momentum reduces this through an accumulated velocity, RMSProp adjusts the learning rate per parameter via averaging the squared gradient, while Adam is a combination of the two. Our results aligned perfectly with this: SGD 5.60% $<$ Momentum 8.66% $<$ RMSProp 9.68% $<$ Adam 9.85%.
11. **What happens when the learning rate is too large?**
    The updates will be larger than the minimum, leading to oscillations, bouncing between valleys or even divergence.

This will make training unstable and might never converge to a good minimum.

12. **What happens when the learning rate is too small?**
    Convergence is very slow because the weight updates do not move far. Figure 8 illustrates this clearly. After 6 epochs, lr 10^{-4} converges to an accuracy of just 5.26% compared to 8.32% for lr
13. **What is the effect of increasing batch size?**
    Larger batches provide lower variance gradients but result in fewer updates per epoch. Accuracy is depicted as falling from 8.49% to 3.74% as the batch size increases from 16 to 64 respectively in plot 9.
14. **Explain stride and padding.**
    Stride indicates the distance that the kernel travels through the input. It is worth mentioning that the bigger stride, the smaller output. Padding refers to adding a margin (usually zeroes) to the image so as to control its output size ("valid" reduces it, "same" keeps it).
15. **Why is MobileNetV2 computationally efficient?**
    It employs depth-wise separable convolutions (much less FLOPs compared to regular convolutions) and inverted residual blocks using linear bottlenecks that ensure the data transported from one block to another remains small. It is significantly lighter compared to VGG and ResNet architectures.
16. **What is depthwise separable convolution?**
    It decomposes the ordinary convolution operation to one with a spatial convolution stage (with one filter per input channel) followed by a 1×1 pointwise convolution stage to combine channels. It requires orders of magnitude fewer parameters/FLOPs than an ordinary convolution with equivalent receptive field.
17. **What is transfer learning?**
    Using the weights/features of a pre-trained model that has learned a big source task (for example, ImageNet) as the basis to learn a new target task (pet breed classification) that is somehow related.
18. **Differentiate feature extraction and fine-tuning.**
    Freezing involves freezing the whole pretrained model and training a new head – quick and risk-free. Meanwhile, fine-tuning entails unfreezing the top layers of the pretrained model and training them in addition to the head at a low learning rate. We observed an increase from 85.74% (freezing) to 87.61% (fine-tuning).
19. **Why is a smaller learning rate generally used during fine-tuning?**
    However, the pre-trained base is already well-optimized for ImageNet features, and using a larger learning rate to train will introduce disruptive changes which will destroy those features (catastrophic forgetting). A smaller learning rate will adapt the features instead.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**
    A single split gives one noisy performance estimate. K-Fold CV averages over K splits, giving both a reliable mean and an SD that shows consistency – separating C1/C2 (mean ≈88%, SD ≈1) from the far less consistent C3/C4 (SD 2.5–2.9).
21. **Why must the test set remain untouched during tuning?**
    Its use during tuning results in implicit adaptation of the models to it (information leakage), thus producing too optimistic accuracy estimation. The fact that it remained untouched up until the single evaluation phase is what makes the test accuracy of 85.85% reliable.
22. **Why should mean and standard deviation both be reported?**
    Mean represents the average performance while SD represents its consistency across different folds. By reporting both mean and SD, we found that C1/C2 was strong and consistent, whereas C3/C4 was poor and very inconsistent.
23. **Is the highest validation accuracy always sufficient to select a model?**
    Not necessarily – fold variance (SD), performance stability of CV on the unaltered test dataset, and computational expense are also relevant factors. C2 scored higher than C1 on average accuracy (88.02 versus 87.96), but the fact that C1 had a smaller SD (0.88 versus 0.99) is a good enough justification.

16. Additional Exercise

Choose two more configurations of learning rate, dropout, batch size and fine-tuning technique. Measure their performance using 5-fold cross-validation and compare the results obtained by the selected configuration (configuration C2: learning rate = 10^{-3}, dropout = 0.5, batch size = 32, mean CV accuracy $88.02\% \pm 0.99$).

Students will have to provide their reasoning whether the chosen configuration is better from the point of accuracy, standard deviation and computation cost.

17. Expected Outcome

This experiment has shown that performance of the CNN is highly dependent on initialization, regularization, optimization, and hyperparameter choices: for the from scratch CNN, initialization and optimization contributed to changes in best validation accuracy of approximately 3-10 percent points, while BatchNorm provided the best results in regularization. The use of transfer learning with frozen and pretrained MobileNetV2 provided much higher

accuracy (85.74feature extraction and 87.61scratch architecture, while the final model selected using 5-fold CV (C2) generalized well, reaching 85.85

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`